Unit 1: OOP Principles and Java Basics

1. **Hello World with a Class and Method:** A simple program demonstrating the basic structure of a Java class and the main method[1].

Java

```
public class HelloWorld {
   public static void main(String[] args) {
      System.out.println("Hello, World!");
   }
}
```

2. **Encapsulation with Getters and Setters:** An example of creating a class with private fields and public methods to control access, a core principle of encapsulation.

Java

```
public class Car {
   private String model;
   private int year;

   public String getModel() {
      return model;
   }

   public void setModel(String model) {
      this.model = model;
   }

   public int getYear() {
      return year;
```

```
    }

    public void setYear(int year) {
        this.year = year;
    }

    public static void main(String[] args) {
        Car myCar = new Car();
        myCar.setModel("Tesla Model S");
        myCar.setYear(2023);
        System.out.println("Car Model: " + myCar.getModel());
    }
}
```

3. **Method Overloading (Polymorphism):** Demonstrates how multiple methods can share the same name but have different parameter lists.

Java

```
class Calculator {
    public int add(int a, int b) {
        return a + b;
    }

    public double add(double a, double b) {
        return a + b;
    }

    public int add(int a, int b, int c) {
```

```
        return a + b + c;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        System.out.println("Sum of two integers: " + calc.add(10, 20));
        System.out.println("Sum of three integers: " + calc.add(10, 20, 30));
        System.out.println("Sum of two doubles: " + calc.add(10.5, 20.5));
    }
}
```

4. **Static Keyword Usage:** A program that uses a static variable to represent a common property and a static method to operate on it, without requiring an object instance[2222].

Java

```
class Employee {
    int id;
    String name;
    static String companyName = "Tech Solutions"; // Static variable

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    static void changeCompany() {
```

```
        companyName = "Global Tech Inc."; // Static method can change static data
    }

    void display() {
        System.out.println(id + " " + name + " " + companyName);
    }
}

public class StaticExample {
    public static void main(String[] args) {
        Employee.changeCompany(); // Calling static method without creating an object
        Employee e1 = new Employee(101, "Alice");
        e1.display();
    }
}
```

5. **Passing Objects as Arguments:** An example showing how changes to an object inside a method affect the original object because the reference is passed by value[3].

Java

```
class Dog {
    String name;
    Dog(String name) { this.name = name; }
}

public class PassByReference {
    public static void changeDogName(Dog d, String newName) {
        d.name = newName; // This changes the original object
```

```
    }

    public static void main(String[] args) {
        Dog myDog = new Dog("Buddy");
        System.out.println("Original Name: " + myDog.name);
        changeDogName(myDog, "Max");
        System.out.println("New Name: " + myDog.name);
    }
}
```

Unit 2: Data Types, Variables, and Operators

6. **Arithmetic and Bitwise Operations without Object Creation:** A program that performs various operations directly within the main method using local variables.

Java

```
public class DirectOperations {
    public static void main(String[] args) {
        int a = 10, b = 5;
        System.out.println("Addition: " + (a + b));
        System.out.println("Subtraction: " + (a - b));
        System.out.println("Bitwise AND: " + (a & b));
        System.out.println("Left Shift: " + (a << 2));
    }
}
```

7. **Arithmetic and Bitwise Operations with Methods and Classes:** A program that encapsulates operations within separate methods and classes, demonstrating better OOP design.

Java

```
class MathOperations {
```

```
    public int add(int a, int b) { return a + b; }
    public int subtract(int a, int b) { return a - b; }
}

class BitwiseOperations {
    public int and(int a, int b) { return a & b; }
    public int leftShift(int a, int b) { return a << b; }
}

public class OOPOperations {
    public static void main(String[] args) {
        MathOperations math = new MathOperations();
        BitwiseOperations bitwise = new BitwiseOperations();

        System.out.println("Addition: " + math.add(20, 10));
        System.out.println("Subtraction: " + math.subtract(20, 10));
        System.out.println("Bitwise AND: " + bitwise.and(20, 10));
        System.out.println("Left Shift: " + bitwise.leftShift(20, 2));
    }
}
```

8. **Type Conversion and Casting:** Demonstrates both implicit widening conversion and explicit narrowing conversion, highlighting potential data loss.

Java

```
public class TypeCasting {
    public static void main(String[] args) {
        // Widening (Implicit)
```

```
        int myInt = 9;
        double myDouble = myInt;
        System.out.println("Widening: " + myDouble);

        // Narrowing (Explicit)
        double largeDouble = 9.78;
        int largeInt = (int) largeDouble;
        System.out.println("Narrowing: " + largeInt);
    }
}
```

9. **Solving a Quadratic Equation:** This program uses Scanner for user input and conditional logic to handle different cases of the discriminant (b2−4ac), showing how to apply mathematical formulas in a program[4].

Java

```
import java.util.Scanner;

public class QuadraticEquation {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter coefficients a, b, and c:");
        double a = sc.nextDouble();
        double b = sc.nextDouble();
        double c = sc.nextDouble();

        double discriminant = b * b - 4 * a * c;

        if (discriminant > 0) {
```

```
            double root1 = (-b + Math.sqrt(discriminant)) / (2 * a);
            double root2 = (-b - Math.sqrt(discriminant)) / (2 * a);
            System.out.println("Real solutions are " + root1 + " and " + root2);
        } else if (discriminant == 0) {
            double root = -b / (2 * a);
            System.out.println("One real solution: " + root);
        } else {
            System.out.println("No real solutions.");
        }
        sc.close();
    }
}
```

10. **Employee Details using Scanner:** A simple program that reads and displays employee details from user input.

Java

```
import java.util.Scanner;

public class EmployeeDetails {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter employee name: ");
        String name = scanner.nextLine();
        System.out.print("Enter employee ID: ");
        int id = scanner.nextInt();
        System.out.print("Enter employee salary: ");
        double salary = scanner.nextDouble();
```

```
        System.out.println("\nEmployee Details:");
        System.out.println("Name: " + name);
        System.out.println("ID: " + id);
        System.out.println("Salary: " + salary);
        scanner.close();
    }
}
```

Unit 3: Control Statements

11. **Prime Number Checker:** A program that takes an integer and prints all prime numbers up to that integer, using nested loops and conditional statements[5].

Java

```
import java.util.Scanner;


public class PrimeNumbers {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int limit = sc.nextInt();


        for (int i = 2; i <= limit; i++) {
            boolean isPrime = true;
            for (int j = 2; j <= Math.sqrt(i); j++) {
                if (i % j == 0) {
                    isPrime = false;
                    break;
```

```
            }
        }
        if (isPrime) {
            System.out.print(i + " ");
        }
    }
    sc.close();
  }
}
```

12. **Fibonacci Sequence (Recursive):** A program that uses a recursive function to calculate the nth Fibonacci number, demonstrating the concept of a method calling itself[6].

Java

```
public class FibonacciRecursive {
  public static int fib(int n) {
    if (n <= 1) {
      return n;
    }
    return fib(n - 1) + fib(n - 2);
  }

  public static void main(String[] args) {
    int n = 10;
    System.out.println("Fibonacci of " + n + " is: " + fib(n));
  }
}
```

13. **Fibonacci Sequence (Non-Recursive):** A program that uses a for loop to compute the nth Fibonacci number, providing a more memory-efficient alternative to recursion.

Java

```
public class FibonacciNonRecursive {
    public static int fib(int n) {
        if (n <= 1) {
            return n;
        }
        int a = 0, b = 1;
        for (int i = 2; i <= n; i++) {
            int next = a + b;
            a = b;
            b = next;
        }
        return b;
    }

    public static void main(String[] args) {
        int n = 10;
        System.out.println("Fibonacci of " + n + " is: " + fib(n));
    }
}
```

14. **Palindrome Number Check:** A program that uses a while loop to reverse a number and then checks if it is equal to the original number[7].

Java

```
import java.util.Scanner;

public class PalindromeCheck {
```

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number to check for palindrome: ");
        int num = sc.nextInt();
        int originalNum = num;
        int reversedNum = 0;

        while (num != 0) {
            int digit = num % 10;
            reversedNum = reversedNum * 10 + digit;
            num /= 10;
        }

        if (originalNum == reversedNum) {
            System.out.println(originalNum + " is a palindrome.");
        } else {
            System.out.println(originalNum + " is not a palindrome.");
        }
        sc.close();
    }
}
```

15. **Grade Calculator using if-else-if:** A program that prompts for marks and uses an if-else-if ladder to assign a grade based on the given criteria[8].

Java

```
import java.util.Scanner;
```

```
public class GradeCalculator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter marks for 3 subjects: ");
        int s1 = sc.nextInt();
        int s2 = sc.nextInt();
        int s3 = sc.nextInt();

        double percentage = (s1 + s2 + s3) / 3.0;
        System.out.println("Percentage: " + percentage);

        if (percentage >= 90) {
            System.out.println("A Grade");
        } else if (percentage >= 70 && percentage < 90) {
            System.out.println("B Grade");
        } else if (percentage >= 40 && percentage < 70) {
            System.out.println("C Grade");
        } else {
            System.out.println("Fail");
        }
        sc.close();
    }
}
```

Unit 4: Arrays

16. **Basic Array Operations:** A program that declares, initializes, and iterates over a single-dimensional array, demonstrating basic array syntax.

Java

```
public class ArrayBasics {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};
        System.out.println("Elements of the array:");

        for (int i = 0; i < numbers.length; i++) {
            System.out.println("Element at index " + i + ": " + numbers[i]);
        }
        // Modify an element
        numbers[2] = 35;
        System.out.println("Modified element at index 2: " + numbers[2]);
    }
}
```

17. **Frequency of Elements in an Array:** A program that counts and displays the frequency of each element in an integer array, using nested loops or a more advanced data structure like a HashMap.

Java

```
public class FrequencyCounter {
    public static void main(String[] args) {
        int[] arr = {1, 2, 8, 3, 2, 2, 5, 1};
        int[] visited = new int[arr.length];
        int count;

        for (int i = 0; i < arr.length; i++) {
            if (visited[i] == 1) continue;
            count = 1;
```

```
        for (int j = i + 1; j < arr.length; j++) {
            if (arr[i] == arr[j]) {
                count++;
                visited[j] = 1;
            }
        }
        System.out.println(arr[i] + " occurs " + count + " times.");
    }
  }
}
```

18. **Matrix Multiplication (Multi-Dimensional Arrays):** A program that multiplies two matrices (2D arrays), demonstrating how to work with nested arrays.

Java

```
public class MatrixMultiplication {
  public static void main(String[] args) {
    int[][] matrix1 = {{1, 2}, {3, 4}};
    int[][] matrix2 = {{5, 6}, {7, 8}};
    int[][] result = new int[2][2];

    for (int i = 0; i < 2; i++) {
      for (int j = 0; j < 2; j++) {
        for (int k = 0; k < 2; k++) {
          result[i][j] += matrix1[i][k] * matrix2[k][j];
        }
        System.out.print(result[i][j] + " ");
      }
```

```
            System.out.println();
        }
    }
}
```

19. **Sorting an Array:** A program that uses Java's built-in Arrays.sort() method to sort an array in ascending order.

Java

```
import java.util.Arrays;


public class ArraySortExample {
    public static void main(String[] args) {
        int[] numbers = {5, 2, 8, 7, 1};
        Arrays.sort(numbers);
        System.out.println("Sorted Array: " + Arrays.toString(numbers));
    }
}
```

20. **Dynamic Array Sizing:** A program that demonstrates how to "dynamically" change the size of an array by creating a new, larger array and copying elements from the original.

Java

```
public class ArrayResizing {
    public static void main(String[] args) {
        int[] oldArray = {1, 2, 3};
        int newSize = 5;
        int[] newArray = new int[newSize];


        System.arraycopy(oldArray, 0, newArray, 0, oldArray.length);
```

```
        System.out.print("Old array: ");
        for (int num : oldArray) System.out.print(num + " ");

        System.out.print("\nNew array: ");
        for (int num : newArray) System.out.print(num + " ");
    }
}
```